

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11
ERROR HANDLING



Disusun Oleh:
Iman Dwi Satrio 105224029

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Praktikum Pemrograman Berorientasi Objek (PBO) pada modul ini berfokus pada pemahaman dan implementasi Error Handling di dalam pemrograman Java sebagai mekanisme pertahanan sistem terhadap error tak terduga (runtime errors). Berbeda dengan pendekatan pengkondisian konvensional (if-else) yang biasanya menggabungkan logika bisnis dengan kontrol kesalahan, mekanisme Error Handling memisahkan kedua aspek tersebut menggunakan blok try-catch-finally. Implementasi studi kasus pada praktikum ini yaitu pada perancangan program simulasi Sistem Reservasi Tiket Kereta Api "JAVA EXPRESS" berbasis pemrograman berorientasi objek yang mengintegrasikan penanganan error berlapis guna menjamin stabilitas aplikasi saat berjalan.

Sistem "JAVA EXPRESS" dirancang menggunakan kombinasi Custom Exception bertipe Checked dan Unchecked Exception guna memvalidasi data bisnis secara spesifik sesuai domain aplikasi. Dua komponen Checked Custom Exception, yaitu kegagalan pencarian rute kereta dan keterbatasan ketersediaan kursi, diimplementasikan dengan mewarisi kelas Exception dasar. Hal ini agar pemanggil fungsi untuk secara eksplisit mendeklarasikan kata kunci throws dan membungkus blok kode melalui penanganan try-catch terurut dari tingkat subkelas yang spesifik. Sementara itu, validasi format Nomor Induk Kependudukan (NIK) penumpang menerapkan Unchecked Exception berbasis RuntimeException untuk mendeteksi programming error atau ketidakpatuhan input seketika.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	kodeKereta	String	Menyimpan identitas atau kode unik dari setiap kereta.
2	namaKereta	String	Menyimpan informasi nama dari kereta.
3	rute	String	Menyimpan informasi stasiun asal dan stasiun tujuan kereta
4	sisakursi	int	Menyimpan jumlah kuantitas kapasitas kursi yang tersedia.
5	jumlah	int	Variabel penampung jumlah kursi yang dipesan.
6	daftarKereta	List	Koleksi dinamis berbasis ArrayList untuk menampung seluruh data objek kereta.
7	nik	String	Variabel untuk menampung 16 digit nomor identitas penumpang.

8	namaPenumpang	String	Parameter input untuk menyimpan nama lengkap dari pemesan tiket.
9	jumlahTiket	int	Parameter input penampung kuantitas tiket yang ingin dipesan.
10	k	Kereta	Variabel penampung objek sementara dalam perulangan for-each
11	sistemReservasi	PusatReservasi	Objek utama sistem reservasi yang diinstansiasi di dalam kelas Main.
12	input	Scanner	Objek resource stream untuk membaca input data dari pengguna via konsol.
13	pilihan	int	Menyimpan angka menu operasi (1-3) yang dipilih oleh pengguna.
14	kode	String	Variabel penampung input kode kereta dari user pada case 2 kelas Main.
15	nama	String	Variabel penampung input nama penumpang dari user pada case 2 kelas Main
16	keretaDipilih	Kereta	Objek penampung sementara saat proses pencarian kecocokan kode kereta.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	Kereta()	Constructor	Menginisialisasi objek model kereta baru yang mencakup kode, nama, rute, dan jumlah kapasitas kursi awal.
2	getKodeKereta()	Functional	Mengambil nilai atribut privat kodeKereta (Getter).
3	getNamaKereta()	Functional	Mengambil nilai atribut privat namaKereta (Getter).
4	getRute()	Functional	Mengambil nilai atribut privat rute (Getter).

5	getSisaKursi()	Functional	Mengambil nilai atribut privat sisaKursi (Getter).
6	kurangiKursi())	Procedural	Mengubah nilai kuantitas sisa kursi kereta dengan cara mengurangnya saat ada pemesanan tiket yang sukses.
7	DataPenumpangTidakValidException()	Constructor	Menginisialisasi objek unchecked custom exception dengan membawa pesan (error message) terkait ketidaksesuaian data NIK.
8	RuteTidakDitemukanException()	Constructor	Menginisialisasi objek checked custom exception dengan membawa pesan error saat kode kereta atau rute tidak ditemukan di sistem.
9	TiketHabisException()	Constructor	Menginisialisasi objek checked custom exception dengan membawa informasi nama kereta dan sisa kapasitas kursi yang tersedia.
10	PusatReservasi()	Constructor	Menginisialisasi objek kontrol reservasi sekaligus memicu pengisian data awal (hardcoded) kereta ke dalam memori.
11	getDaftarKereta()	Functional	Mengambil seluruh koleksi objek Kereta yang terdaftar di dalam ArrayList.
12	pesanTiket()	Procedural	Memproses logika bisnis reservasi tiket, melakukan validasi NIK (memicu DataPenumpangTidakValidException), mencocokkan kode rute (throws RuteTidakDitemukanException), dan memeriksa ketersediaan kursi (throws TiketHabisException).
13	main()	Procedural	Program utama (main entry point) untuk menjalankan seluruh simulasi antarmuka

			berbasis CLI dan mengendalikan alur try-catch.
--	--	--	--

IV. Dokumentasi dan Pembahasan Code

Bagian ini akan membahas logika pemrograman yang diimplementasikan.



```

1  public class Kereta {
2      private String kodeKereta;
3      private String namaKereta;
4      private String rute;
5      private int sisaKursi;
6
7      public Kereta(String kodeKereta, String namaKereta, String rute, int sisaKursi) {
8          this.kodeKereta = kodeKereta;
9          this.namaKereta = namaKereta;
10         this.rute = rute;
11         this.sisaKursi = sisaKursi;
12     }
13
14     public String getKodeKereta() { return kodeKereta; }
15     public String getNamaKereta() { return namaKereta; }
16     public String getRute() { return rute; }
17     public int getSisaKursi() { return sisaKursi; }
18
19     public void kurangiKursi(int jumlah) {
20         this.sisaKursi -= jumlah;
21     }
22 }
23

```

Penjelasan: Kelas Kereta bertindak sebagai kelas model data yang mengimplementasikan salah satu konsep fundamental Pemrograman Berorientasi Objek, yaitu Enkapsulasi.

1. Deklarasi Atribut Ber-modifier Private (Baris 2-5): Seluruh variabel (kodeKereta, namaKereta, rute, dan sisaKursi) dideklarasikan menggunakan akses modifier private. Hal ini bertujuan untuk menyembunyikan status atau detail data internal dari jangkauan akses langsung oleh kelas eksternal, sehingga integritas data objek tetap terjaga dari modifikasi ilegal secara eksternal.
2. Konstruktor Berparameter (Baris 7-12): Konstruktor public Kereta(...) menerima empat parameter input untuk melakukan inisialisasi awal nilai state objek sesaat setelah objek tersebut diciptakan menggunakan kata kunci new. Penggunaan kata kunci this pada baris 8 hingga 11 wajib diterapkan guna membedakan serta merujuk langsung kepada variabel instansiasi milik kelas (instance variable) dari variabel parameter dengan penamaan yang identik.
3. Metode Getter (Baris 14-17): Karena atribut kelas bersifat tersembunyi (private), kelas ini menyediakan metode fungsional Getter (getKodeKereta, getNamaKereta, getRute, getSisaKursi) dengan hak akses public. Metode ini hanya memberikan akses baca (read-only) bagi kelas luar yang membutuhkan data informasi tersebut.

4. Metode Prosedural kurangiKursi (Baris 19-21): Metode ini dirancang untuk memanipulasi kapasitas sisa kursi secara aman di dalam objek. Operasi penugasan majemuk pengurangan (--) pada baris 20 mengeksekusi logika bisnis pemotongan kuantitas kursi berdasarkan argumen jumlah tiket yang dikirim di parameter. Metode ini yang nantinya akan dilindungi oleh mekanisme pengecekan batas error pada kelas pengolah reservasi.

```
1 public class DataPenumpangTidakValidException extends RuntimeException {  
2     public DataPenumpangTidakValidException(String message) {  
3         super(message);  
4     }  
5 }  
6
```

Penjelasan: Kelas `DataPenumpangTidakValidException` dirancang sebagai Custom Exception untuk mendeteksi kesalahan format pada data masukan pengguna sebelum diproses lebih jauh oleh sistem.

1. Pewarisan `RuntimeException` (Baris 1): Penggunaan kata kunci `extends RuntimeException` menandakan bahwa kelas error buatan ini dikategorikan ke dalam `Unchecked Exception`. Struktur ini memberikan fleksibilitas tinggi bagi arsitektur kode karena compiler Java tidak mewajibkan penanganan error secara eksplisit menggunakan blok `try-catch` atau deklarasi `throws` pada tanda tangan metode pemanggilnya.
2. Mekanisme Konstruktor dan Pengiriman Pesan (Baris 2-4): Konstruktor kelas menerima satu buah parameter string berupa `message` yang membawa informasi detail atau deskripsi spesifik mengenai penyebab terjadinya error. Di dalam konstruktor, pemanggilan metode `super(message)` pada baris 3 berfungsi untuk meneruskan pesan galat tersebut ke konstruktor superclass (`RuntimeException`). Hal ini menjamin bahwa saat error ini dilempar (`thrown`), pesan kesalahan dapat ditangkap dan ditampilkan ke konsol pengguna menggunakan metode bawaan seperti `.getMessage()`.

```
1 public class RuteTidakDitemukanException extends Exception {  
2     public RuteTidakDitemukanException(String message) {  
3         super(message);  
4     }  
5 }  
6
```

Penjelasan: Kelas `RuteTidakDitemukanException` dirancang sebagai Custom Exception untuk mendeteksi kondisi ketika pengguna memasukkan kode kereta yang tidak terdaftar di dalam basis data memori sistem.

1. Pewarisan Kelas Exception (Baris 1): Penggunaan kata kunci `extends Exception` secara eksplisit mengategorikan kelas galat buatan ini ke dalam Checked Exception. Berbeda dengan Unchecked Exception, arsitektur compiler Java memberlakukan aturan di mana setiap metode yang berpotensi melempar galat ini wajib mendeklarasikannya pada method signature menggunakan kata kunci `throws`, atau membungkusnya secara langsung menggunakan blok try-catch di sisi pemanggil fungsi.
2. Penerusan Informasi Galat (Baris 2-4): Konstruktor kelas menerima parameter berupa string message. Melalui instruksi `super(message)` pada baris 3, parameter tersebut dikirimkan langsung ke konstruktor utama kelas induk (Exception). Hal ini memastikan bahwa rantai pelacakan kesalahan tetap terjaga dengan baik, sehingga sistem dapat memberikan pesan umpan balik yang informatif dan presisi mengenai rute atau kode kereta apa yang gagal ditemukan oleh sistem.



```
1 public class TiketHabisException extends Exception {
2     private String namaKereta;
3     private int sisaKursi;
4
5     public TiketHabisException(String namaKereta, int sisaKursi) {
6         super("Tiket gagal dipesan! Kursi tidak mencukupi.");
7         this.namaKereta = namaKereta;
8         this.sisaKursi = sisaKursi;
9     }
10
11     public String getNamaKereta() {
12         return namaKereta;
13     }
14
15     public int getSisaKursi() {
16         return sisaKursi;
17     }
18 }
19
```

Penjelasan: Kelas `TiketHabisException` dirancang sebagai Custom Checked Exception yang memiliki karakteristik dengan membawa status atau data state kontekstual dari objek yang mengalami error.

1. Karakteristik Checked Exception dan Variabel Enkapsulasi (Baris 1-3): Kelas ini mewarisi kelas Exception (`extends Exception`) sehingga dikategorikan sebagai Checked Exception yang mewajibkan penanganan eksplisit saat kompilasi program. Berbeda dengan kelas galat sebelumnya, kelas ini memiliki dua buah atribut privat, yaitu `namaKereta` dan `sisaKursi`. Keberadaan

variabel ini berfungsi untuk menyimpan informasi riil mengenai status kereta saat transaksi pemesanan tiket tersebut ditolak oleh sistem.

2. Inisialisasi Berlapis pada Konstruktor (Baris 5-9): Konstruktor kelas menerima dua parameter, yaitu nama kereta dan jumlah sisa kursi yang tersedia di memori. Pada baris 6, instruksi `super(...)` dipanggil dengan argumen string untuk mengeset cetak pesan error utama pada kelas induk. Selanjutnya, pada baris 7 dan 8, parameter input dipetakan langsung ke variabel instansiasi objek menggunakan kata kunci `this` agar data kontekstual tersebut melekat pada objek pengecualian yang sedang dibuat.
3. Metode Getter (Baris 11-17): Untuk mendukung prinsip enkapsulasi data, kelas menyediakan dua metode fungsional pengakses publik, yaitu `getNamaKereta()` dan `getSisaKursi()`. Penyediaan metode Getter pada sebuah kelas exception agar blok penangkap kesalahan (`catch block`) di program utama untuk mengekstrak informasi detail internal galat dan menampilkan solusi alternatif yang interaktif serta informatif kepada pengguna (contoh: memberi tahu sisa kursi yang sebenarnya tersedia).

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class PusatReservasi {
5     private List<Kereta> daftarKereta;
6
7     public PusatReservasi() {
8         daftarKereta = new ArrayList<>();
9         daftarKereta.add(new Kereta("K91", "Argo Bromo", "JKT - SBY", 50));
10        daftarKereta.add(new Kereta("K92", "Parahyangan", "JKT - BDG", 15));
11    }
12
13    public List<Kereta> getDaftarKereta() {
14        return daftarKereta;
15    }
16
17    public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket) throws RuteTidakDitemukanException, TiketHabisException {
18        if (nik.length() != 16 || !nik.matches("[0-7]+")) {
19            throw new DataPenumpangTidakValidException("Error: NIK harus tepat 16 digit dan hanya berisi angka!");
20        }
21
22        Kereta keretaDipilih = null;
23        for (Kereta k : daftarKereta) {
24            if (k.getKodeKereta().equalsIgnoreCase(kodeKereta)) {
25                keretaDipilih = k;
26                break;
27            }
28        }
29
30        if (keretaDipilih == null) {
31            throw new RuteTidakDitemukanException("Error: Kode kereta atau rute " + kodeKereta + " tidak ditemukan!");
32        }
33
34        if (jumlahTiket > keretaDipilih.getSisaKursi()) {
35            throw new TiketHabisException(keretaDipilih.getNamaKereta(), keretaDipilih.getSisaKursi());
36        }
37
38        keretaDipilih.kurangiKursi(jumlahTiket);
39        System.out.println("RESERVASI BERHASIL");
40        System.out.println("Nama Penumpang: " + namaPenumpang);
41        System.out.println("Kereta: " + keretaDipilih.getNamaKereta());
42        System.out.println("Rute: " + keretaDipilih.getRute());
43        System.out.println("Jumlah Tiket: " + jumlahTiket + " kursi");
44    }
45 }
46
```

Penjelasan: Kelas `PusatReservasi` bertindak sebagai komponen Controller yang mengatur manipulasi data objek kereta di dalam memori dan menegakkan aturan bisnis melalui `throw exception` terstruktur.

1. Inisialisasi Data Menggunakan Collections Framework (Baris 7-11): Atribut `daftarKereta` dideklarasikan menggunakan tipe antarmuka `List` dan diinstansiasi menggunakan kelas konkret `ArrayList`. Struktur data ini dipilih karena sifatnya yang dinamis dalam menampung data objek di memori. Di dalam konstruktor kelas (baris 7), sistem langsung mendaftarkan dua buah objek `Kereta` awal secara berurutan menggunakan metode bawaan `.add()`.

2. Validasi NIK Penumpang dengan Unchecked Exception (Baris 18-20): Sebelum memproses data pemesanan, metode pesanTiket melakukan validasi format NIK pengguna. Evaluasi dilakukan menggunakan kombinasi pengecekan panjang karakter (`.length() != 16`) dan ekspresi reguler (`.matches("[0-9]+")`). Apabila salah satu kondisi tidak terpenuhi, program akan secara paksa melempar `DataPenumpangTidakValidException` menggunakan kata kunci `throw` (baris 19). Karena merupakan Unchecked Exception, metode tidak memerlukan deklarasi `throws` untuk exception ini.
3. Pencarian Data dan Checked Exception Rute (Baris 22-32): Sistem melakukan pencarian kereta menggunakan perulangan `for-each` (baris 23). Metode `.equalsIgnoreCase()` digunakan untuk mencocokkan kode kereta tanpa mempedulikan huruf kapital. Jika setelah perulangan selesai variabel `keretaDipilih` tetap bernilai `null` (baris 30), program akan menghentikan eksekusi dengan melempar `RuteTidakDitemukanException`. Exception ini didaftarkan menggunakan kata kunci `throws` di baris 17 karena termasuk dalam rumpun Checked Exception.
4. Validasi Ketersediaan Kursi (Baris 34-43): Tahap validasi terakhir memeriksa ketersediaan fisik kursi kereta dengan membandingkan parameter `jumlahTiket` terhadap nilai kembalian `.getSisaKursi()`. Jika permintaan melebihi kapasitas, `TiketHabisException` dilempar dengan menyertakan argumen nama kereta dan kapasitas tersisa (baris 35). Apabila seluruh rangkaian pengecekan berhasil dilewati, baris 38 akan menjalankan `.kurangiKursi()` dan mencetak bukti struk reservasi tiket ke konsol.

```

1  import java.util.InputMismatchException;
2  import java.util.Scanner;
3
4  public class Main {
5      public static void main(String[] args) {
6          PusatReservasi sistemReservasi = new PusatReservasi();
7          Scanner input = new Scanner(System.in);
8          int pilihan = 0;
9
10         do {
11             try {
12                 System.out.println("MENU UTAMA:");
13                 System.out.println("1. Lihat Jadwal");
14                 System.out.println("2. Pesan Tiket");
15                 System.out.println("3. Keluar");
16                 System.out.print("Pilih menu (1-3): ");
17                 pilihan = input.nextInt();
18                 input.nextLine();
19
20                 switch (pilihan) {
21                     case 1:
22                         System.out.printf("%-10s %-15s %-15s %-10s\n", "Kode", "Nama Kereta", "Rute", "Sisa Kursi");
23                         for (Kereta k : sistemReservasi.getDaftarKereta()) {
24                             System.out.printf("%-10s %-15s %-15s %-10d\n", k.getKodeKereta(), k.getNamaKereta(), k.getRute(), k.getSisaKursi());
25                         }
26                         break;
27                     case 2:
28                         System.out.print("Masukkan Kode Kereta: ");
29                         String kode = input.nextLine();
30                         System.out.print("Masukkan NIK (16 digit): ");
31                         String nik = input.nextLine();
32                         System.out.print("Masukkan Nama Penumpang: ");
33                         String nama = input.nextLine();
34                         System.out.print("Masukkan Jumlah Tiket: ");
35                         int jumlah = input.nextInt();
36                         input.nextLine();
37                         sistemReservasi.pesanTiket(kode, nik, nama, jumlah);
38                         break;
39                     case 3:
40                         System.out.println("Keluar dari sistem...");
41                         break;
42                     default:
43                         System.out.println("Pilihan tidak valid! Silakan pilih menu 1, 2, atau 3.");
44                 }
45             } catch (InputMismatchException e) {
46                 System.out.println("Error: Input harus berupa angka!");
47                 input.nextLine();
48             } catch (DataPenumpangTidakValidException e) {
49                 System.out.println(e.getMessage());
50             } catch (RuteTidakDitemukanException e) {
51                 System.out.println(e.getMessage());
52             } catch (TiketHabisException e) {
53                 System.out.println(e.getMessage());
54                 System.out.println("Info Kereta: " + e.getNamaKereta());
55                 System.out.println("Sisa Tersedia: " + e.getSisaKursi() + " kursi");
56             }
57         } while (pilihan != 3);
58     }
59 }
60

```

Penjelasan: Kelas Main bertindak sebagai titik mulai program yang mengintegrasikan seluruh kelas model, controller, dan exception ke dalam satu kesatuan alur program. Fokus utama kelas ini adalah penerapan Multi-catch Block Error Handling untuk menjaga stabilitas sistem.

1. Baris 10-57: Seluruh menu operasi dibungkus di dalam perulangan do-while yang dikombinasikan dengan blok try-catch. Struktur penanganan error yang berada di dalam looping ini menjamin aplikasi memiliki sifat atau apabila terjadi kesalahan input atau kegagalan transaksi, aplikasi tidak akan mendadak berhenti (crash), melainkan menangkap galat tersebut, menampilkan pesan informatif, dan kembali menyajikan menu utama.
2. Pencegahan Infinite Loop pada Input Mismatch (Baris 45-47): Ketika pengguna salah memasukkan tipe data (misalnya mengetik huruf pada menu yang meminta angka), objek Scanner akan melemparkan InputMismatchException bawaan Java (baris 45). Token masukan yang salah tersebut akan tertahan di dalam memori buffer Scanner. Oleh karena itu, instruksi input.nextLine(); pada baris 46 wajib dieksekusi di dalam blok catch untuk membersihkan sisa buffer. Tanpa baris tersebut, program akan terjebak dalam perulangan tanpa akhir (infinite loop) karena terus-menerus membaca token salah yang sama.

3. Baris 48-56: Program menerapkan penanganan kesalahan berlapis berdasarkan jenis exception yang dipicu oleh kelas `PusatReservasi`.
 - a. `Unchecked Exception (DataPenumpangTidakValidException)`: Ditangkap di baris 48 untuk menginfokan format NIK yang salah.
 - b. `Checked Exception (RuteTidakDitemukanException)`: Ditangkap di baris 50 untuk menginfokan kesalahan input kode rute kereta.
 - c. `TiketHabisException`: Ditangkap di baris 52. Blok penangkap ini memanfaatkan arsitektur khusus dari eksepsi buatan. Melalui pemanggilan metode khusus `.getNamaKereta()` (baris 54) dan `.getSisaKursi()` (baris 55), program dapat menyajikan data real-time dari memori mengenai nama kereta serta sisa kursi riil yang tersedia agar pengguna dapat melakukan penyesuaian pesanan pada transaksi berikutnya.

V. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada sistem reservasi tiket kereta api "JAVA EXPRESS", dapat disimpulkan bahwa penerapan arsitektur Error Handling secara terstruktur menggunakan blok try-catch mampu meningkatkan ketahanan aplikasi terhadap error waktu berjalan (fault tolerance). Integrasi antara Custom Checked Exception (`RuteTidakDitemukanException` dan `TiketHabisException`) dengan `Unchecked Exception (DataPenumpangTidakValidException)` dapat efektif dalam memisahkan logika bisnis utama dari kendali penanganan kesalahan di dalam memori lokal. Selain itu, blok penanganan exception di dalam siklus perulangan antarmuka CLI (Command Line Interface) telah menjaga stabilitas sistem agar aplikasi tidak mengalami penghentian paksa (crash) saat pengguna melakukan kesalahan masukan data.

VI. Daftar Pustaka

Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects* (11th ed.). Pearson.

Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). *Modul Praktikum Pemrograman Berorientasi Objek: Error Handling*. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.